



Other Data Models



Content

- Introduction
- Object-Oriented Database
- XML Database
- NoSQL

Introduction

- Changes in the database
 - Network, hierarchical, relational have been successful in traditional business database applications
 - Complex databases applications
 - CAD/CAM (Computer Aid Design/Computer Aid Manufacturing)
 - Telecommunication
 - GIS (Geographic Information Systems)
 - Web

Introduction

- Newer applications require characteristics
 - More complex structures for stored objects
 - New data types for storing
 - Graphic images, video clips, sound waveforms
 - Large document, large object (LOB)
 - Longer duration transactions
 - Nonstandard operations
- Lead to the development of additional *semantic data modeling* concepts

Content

- Introduction
- **Object-Oriented database**
- XML Database
- NoSQL

Object-Oriented Database

- Appeared in 1980's
 - Experimental prototypes
 - ORION – Microelectronics and Computer Technology
 - OPENOODM – Texas Instruments
 - IRIS – Hewlett Packard Labs
 - ODE – AT&T Bell Labs
 - ENCORE/ObServer – Brown University
 - Commercial systems
 - Gemstone/OPAL – Gemstone system
 - ONTOS – Objectivity Inc.
 - ObjectStore – Object Design
 - ARDENT – Ardent Software
 - POET – Poet Software

Object-Oriented Database

- Standardized by Object Data Management Group
 - ODMG 1.0 (1993)
 - ODMG 2.0 (1997)
 - ODMG 3.0 (1999)
- Adopted many of concepts from OOPL
 - Classes, object, extent, object identity
 - Method
 - Inheritance
 - Class hierarchies
 - ...

Object-Oriented database

- 2 approaches of the object model
 - “Pure” object-oriented
 - ODL (Object Definition Language)
 - * A standardized language for specifying the structure of database in object-oriented terms
 - OQL (Object Query Language)
 - * A standardized query language for object-oriented database
 - * Combines SQL with object-oriented programming paradigm to operate on data described in ODL
 - Object-relational
 - Incorporate many ideas found in ODL
 - Relational model has been extended to object-relational model
 - * Allowing more complex types for tuples, attributes

Example – ODL

```
class DEPARTMENT (extent DEPARTMENTS key DNUMBER) {  
    attribute string DNAME;  
    attribute string DNUBMER;  
    relationship Set<EMPLOYEE> HAS_EMPLOYEES  
        inverse EMPLOYEE::WORKS_FOR;  
    integer NO_EMP() raises (noEmployeeFound);  
    void EMP_FNAME(out Set<string>);  
}
```

Example – ODL

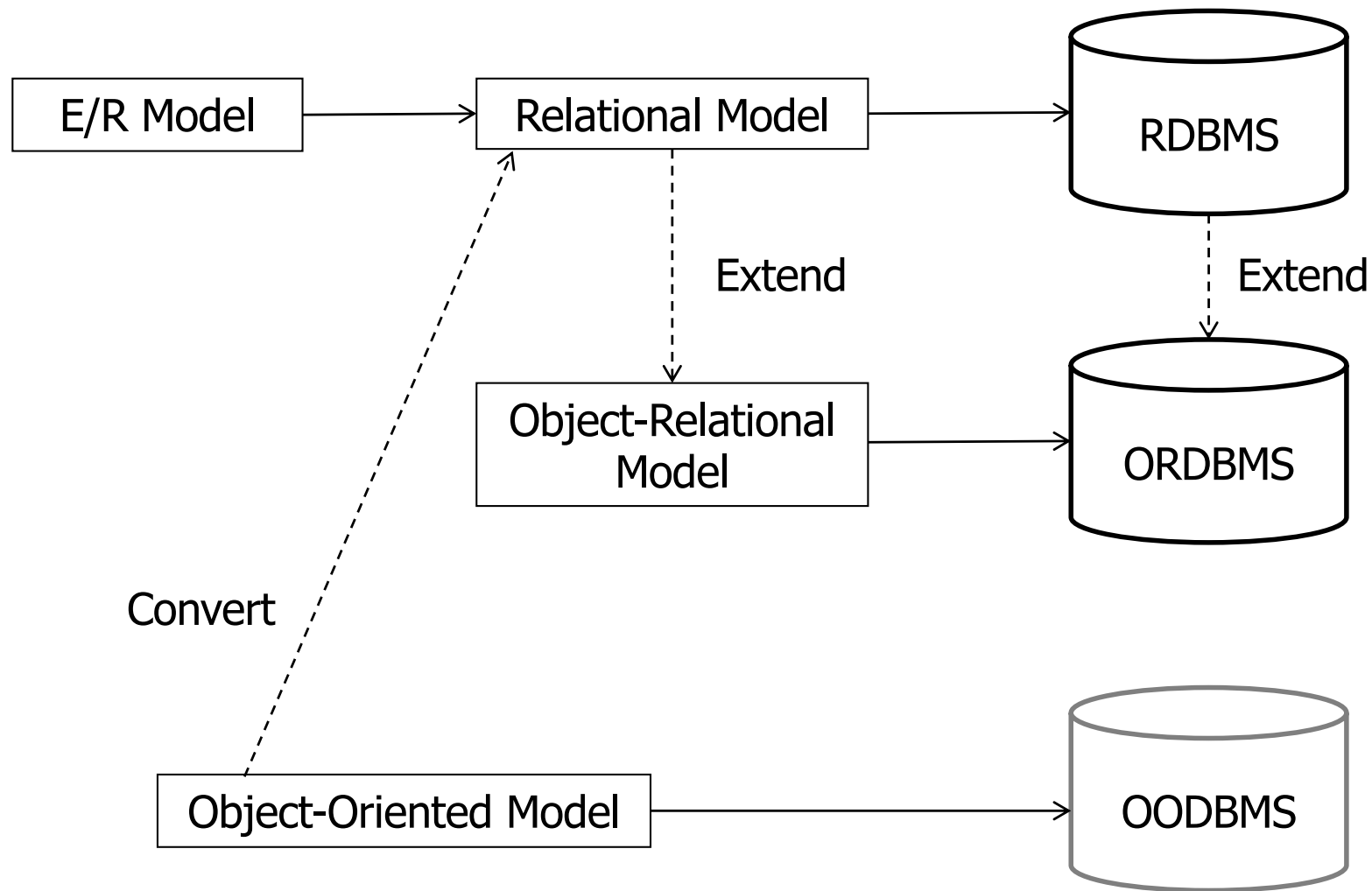
```
class EMPLOYEE ( extent EMPLOYEES key ID) {  
    attribute string LNAME;  
    attribute string MNIT;  
    attribute string FNAME;  
    attribute string ID;  
    attribute Struct DMY  
        {integer DATE, integer MONTH, integer YEAR} BIRTHDATE;  
    attribute string ADDRESS;  
    attribute enum GENDER {male, female} SEX;  
    attribute float SALARY;  
    relationship DEPARTMENT WORKS_FOR  
        inverse DEPARTMENT::HAS_EMPLOYEES;  
}
```

Example – OQL

- Find FName and LName of employees who are working for the department 'Research'

```
SELECT e.FNAME, e.LNAME  
FROM   DEPARTMENTS d, d.HAS_EMPLOYEES e  
WHERE  d.DNAME = "Research"
```

Object-Oriented database



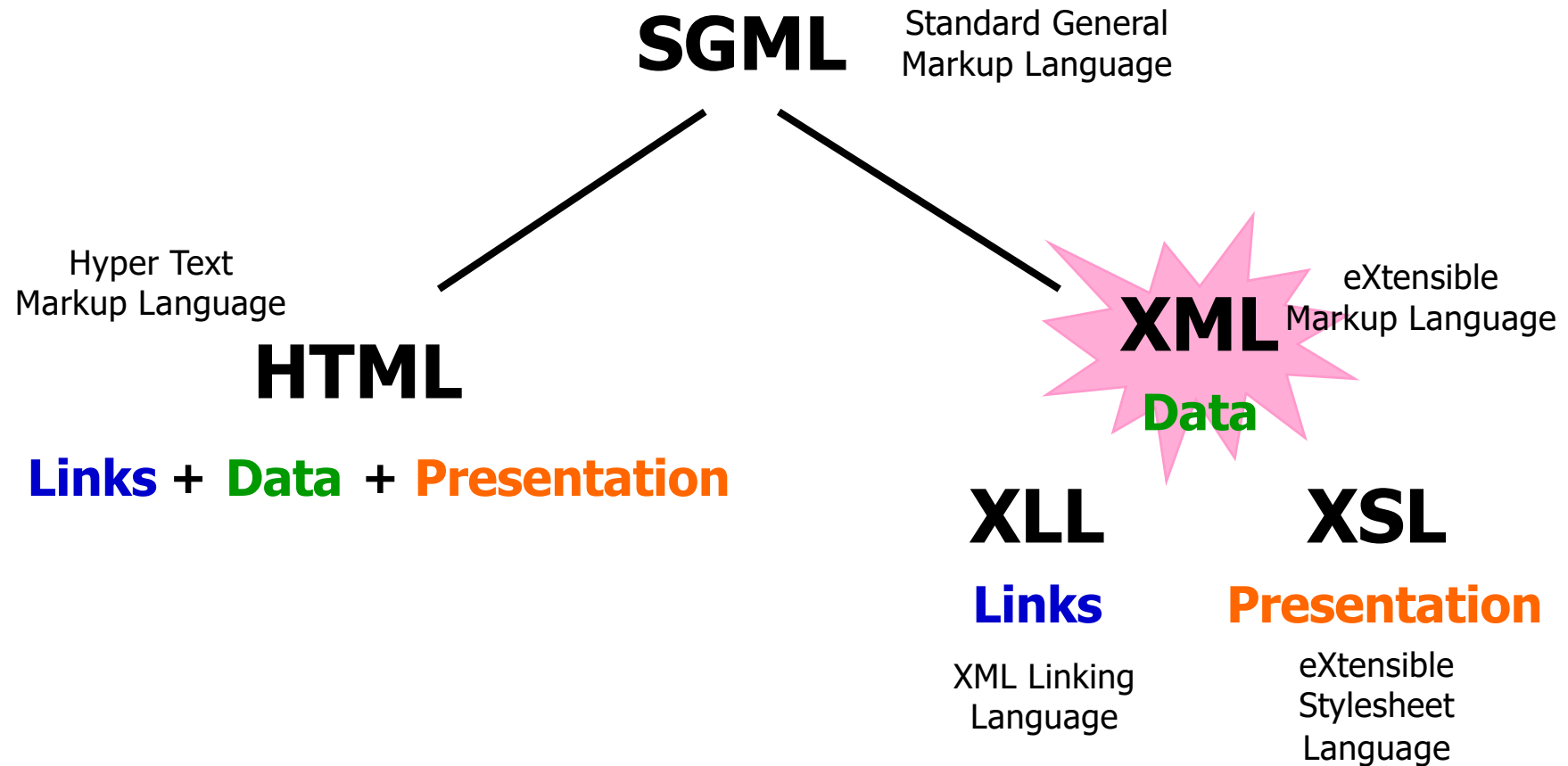
Content

- Introduction
- Object-Oriented database
- **XML database**
- NoSQL

Introduction

- XML
 - e**X**tensible **M**arkup **L**anguage
 - A tag-based notation for “marking” documents
- Appeared 1996
- Standardized by W3C (World Wide Web Consortium)
 - XML 1.0 (1998)
 - XML 2.0 (2000)

Introduction



Introduction

- 3 models
 - Data (structured)
 - Document (unstructured)
 - Hybrid (semi-structured)

- Management XML data or XML document
 - Store, access, search, modify
 - XML database management system
 - XML-Enable database
 - Native XML database

Introduction

- XML-Enable database
 - Support for storing XML data
 - Divide XML document into
 - Tuples (Relational DBMS)
 - Objects (Object-Oriented DBMS)

- Native XML database
 - Support for storing XML document
 - The original structure of documents is preserved

Introduction

- However
 - Index
 - Data security
 - Transaction management
 - Multi-users
 - Integrity constraints

XML document

- XML document includes
 - Element
 - Attribute
- Components
 - DTD (Document Type Definition)
 - XML Schema
 - Describe the structure of an XML document
 - Implement constraints for data
- Query
 - XPath
 - XQuery

Example

<?xml version="1.0" encoding="utf-8" ?>

<memo>

Element

<to>Phuong</to>

<from>Trang</from>

<date>24 December 2005</date>

<subject>Notice</subject>

<text>Please prepare the document carefully.</text>

</memo>

Open tag

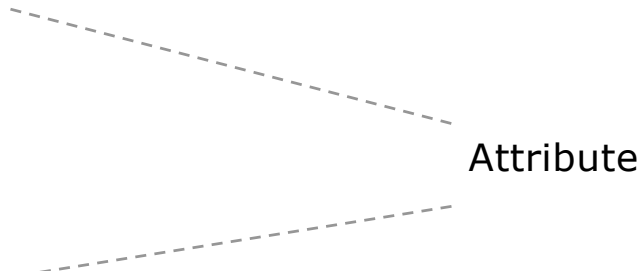
Data

Close tag

Example

```
<?xml version="1.0" encoding="utf-8" ?>  
<?xml-stylesheet type="text/xsl" href="purchaseOrder.xslt"?>
```

```
<PurchaseOrder id="2001" customer="Tom">  
  <Date>  
    <Day> 10 </Day>  
    <Month> June </Month>  
    <Year> 1999 </Year>  
  </Date>  
  <Item name="Firestone Tire" cost="2000.00">  
    <Quantity> 50 </Quantity>  
  </Item>  
  <Item name="Goodyear Tire" cost="8000.00">  
    <Quantity> 200 </Quantity>  
  </Item>  
  <Payment> 40% </Payment>  
  <Payment> 60% </Payment>  
</PurchaseOrder>
```



Attribute

Example – XML

memo.xml

```
<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE memo [
    <!ELEMENT memo (to, from, date, subject?, text+)>
    <!ELEMENT to (#PCDATA)>
    <!ELEMENT from (#PCDATA)>
    <!ELEMENT date (#PCDATA)>
    <!ELEMENT subject (#PCDATA)>
    <!ELEMENT text (#PCDATA)>
]>
<memo>
    <to>Phuong</to>
    <from>Trang</from>
    <date>Ngày 24 tháng 12 năm 2005</date>
    <subject>Bao cao</subject>
    <text>Chuan bi tai lieu ky luong de bao cao</text>
</memo>
```

Example – XML & DTD

memo.dtd

```
<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE memo [
    <!ELEMENT memo (to, from, date, subject?, text+)>
    <!ELEMENT to (#PCDATA)>
    <!ELEMENT from (#PCDATA)>
    <!ELEMENT date (#PCDATA)>
    <!ELEMENT subject (#PCDATA)>
    <!ELEMENT text (#PCDATA)>
]>
```

memo.xml

```
<?xml version="1.0" encoding="utf-8" ?>
<!DOCTYPE memo SYSTEM "memo.dtd">
<memo>
    <to>Phuong</to>
    <from>Trang</from>
    <date> 24 December 2005 </date>
    <subject>Notice</subject>
    <text>Please prepare the document carefully.</text>
</memo>
```

Example – DTD

```
<!DOCTYPE purchaseOrder [  
    <!ELEMENT purchaseOrder (date, item+, payment*)>  
    <!ELEMENT date (day, month, year)>  
    <!ELEMENT day (#PCDATA)>  
    <!ELEMENT month (#PCDATA)>  
    <!ELEMENT year (#PCDATA)>  
    <!ELEMENT item (name, cost)>  
    <!ELEMENT payment (#PCDATA)>  
    <!ATTLIST purchaseOrder  
        id CDATA #REQUIRED  
        customer CDATA #REQUIRED >  
    <!ATTLIST item  
        name CDATA #REQUIRED  
        cost CDATA #REQUIRED >  
>
```

Example – XML schema

```
<?xml version="1.0" encoding="utf-8" ?>
<xs:schema xmlns:xs=" http://www.w3.org/2005/XMLSchema"
  targetNamespace="http://www.w3schools.com"
  xmlns="http://www.w3schools.com">
  <xs:element name="memo">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="to" type="xs:string"/>
      <xs:element name="from" type="xs:string"/>
      <xs:element name="date" type="xs:string"/>
      <xs:element name="subject" type="xs:string"/>
      <xs:element name="text" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
</xs:schema>
```

Example – Constraint

■ Domain

```
<xs:element name="car">
  <xs:simpleType>
    <xs:restriction base="xs:string">
      <xs:enumeration value="Audi"/>
      <xs:enumeration value="Golf"/>
      <xs:enumeration value="BMW"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>
```

XML Schema – Constraint

■ Domain

```
<xs:element name="password">  
  <xs:simpleType>  
    <xs:restriction base="xs:string">  
      <xs:pattern value="[a-zA-Z0-9]{8}"/>  
    </xs:restriction>  
  </xs:simpleType>  
</xs:element>
```

Example – XPath

/purchaseOrder

/purchaseOrder/item

/purchaseOrder/item [cost gt 2000.00]/name

//item

Example – XQuery

```
FOR    $x IN doc(purchaseOrder.xml)/purchaseOrder/item
WHERE  $x/cost gt 2000.00
RETURN <res> $x/name, $x/cost </res>
```

Content

- Introduction
- Object-Oriented database
- XML database
- **NoSQL database**

NoSQL database

- “Not only SQL”
 - Not use the tabular relations as relational databases
 - Has known in 1998
 - Lightweight, open-source relational database
 - Not use the SQL interface
 - In 2009 + Web 2.0
 - Open-source distributed database
 - Non-relational
- The next generation database [nosql-database.org]
 - Non-relational, distributed, open-source
 - Horizontally scalable

Why NoSQL database?

- Limitations of relational database technology in *modern web applications*
 - Google, Facebook, LinkedIn, Twitter...
- A rise in *the volume of data* stored about users, objects and products
 - Millions of different users a day
 - Capturing and using big data
 - Personal user information, social graphs, user-generated content, machine logging data, sensor-generated data
 - Internet of things
 - Devices are connected to the Internet
 - Data on environment, location, movement, temperature, weather... from sensors

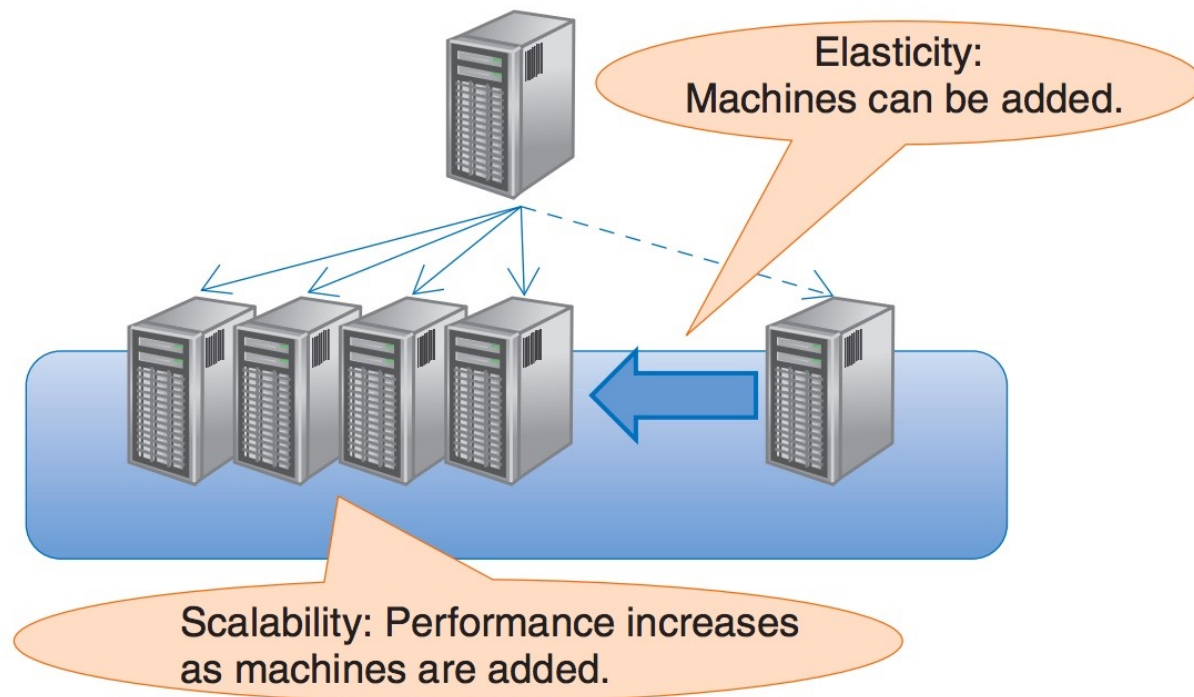
Characteristics

- Scale-out
- Replication
- Flexible data structure

Characteristics

■ Scale-out

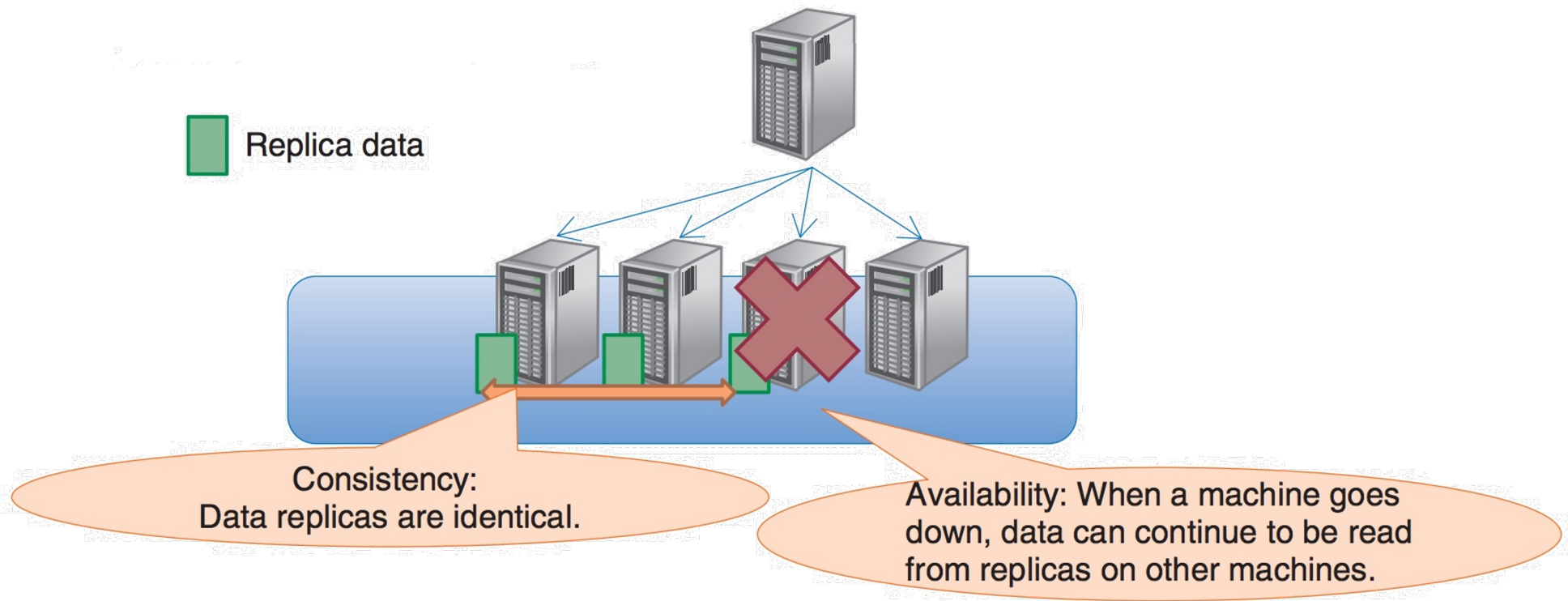
- Achieving high performance by using many general-purpose machines in a distributed manner



Characteristics

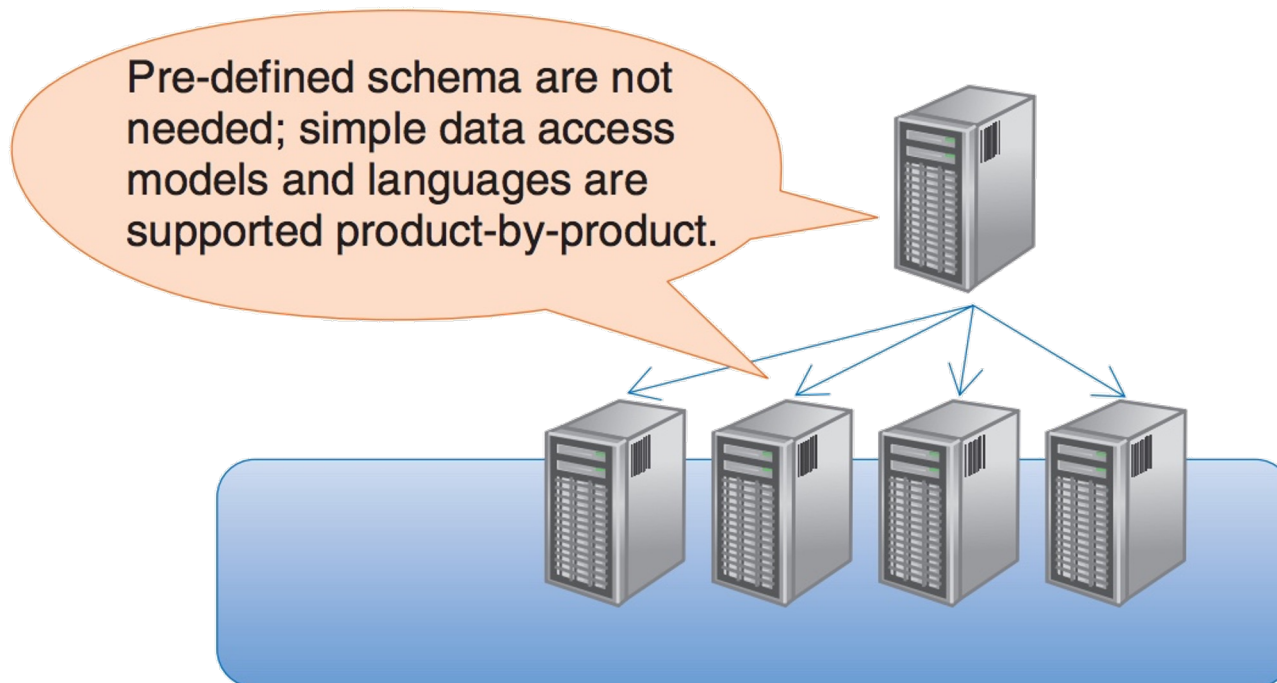
■ Replication

- The copying of data to achieve data redundancy and load distribution



Characteristics

- Flexible data structure
 - Data can be stored with various different structures in the same table



Compatible with various data formats: images (binary), tweets, geographical data, etc.

NTT Technical Report

Categories

- Key-Value
 - Eg. Amazon DynamoDB, Google BigTable
- Column-oriented
 - Eg. Google BigTable, Cassandra
- Document-oriented
 - Eg. MongoDB, Apache CouchDB, Amazon SimpleDB
- Graph
 - Eg. Neo4J, HyperGraphDB

<http://nosql-database.org/>

Key-Value

- Every single item in the database
 - An attribute name (key)
 - Its value

ID	First Name	Last Name
1	Samantha	Jones
2	James	Bond
3	James	Kirk

Key: 1	ID: sj	First Name: Sam
-----------	--------	-----------------

Key: 2	Email: jb@gmail.com	Location: London	Age: 37
-----------	---------------------	------------------	---------

Key: 3	Facebook ID: jkirk	Password: xxx	Name: James
-----------	--------------------	---------------	-------------

Column-oriented

- Data are store as columns, instead of rows

Table

Country	Product	Sales
US	Alpha	3.000
US	Beta	1.250
JP	Alpha	700
UK	Alpha	450

Row Store

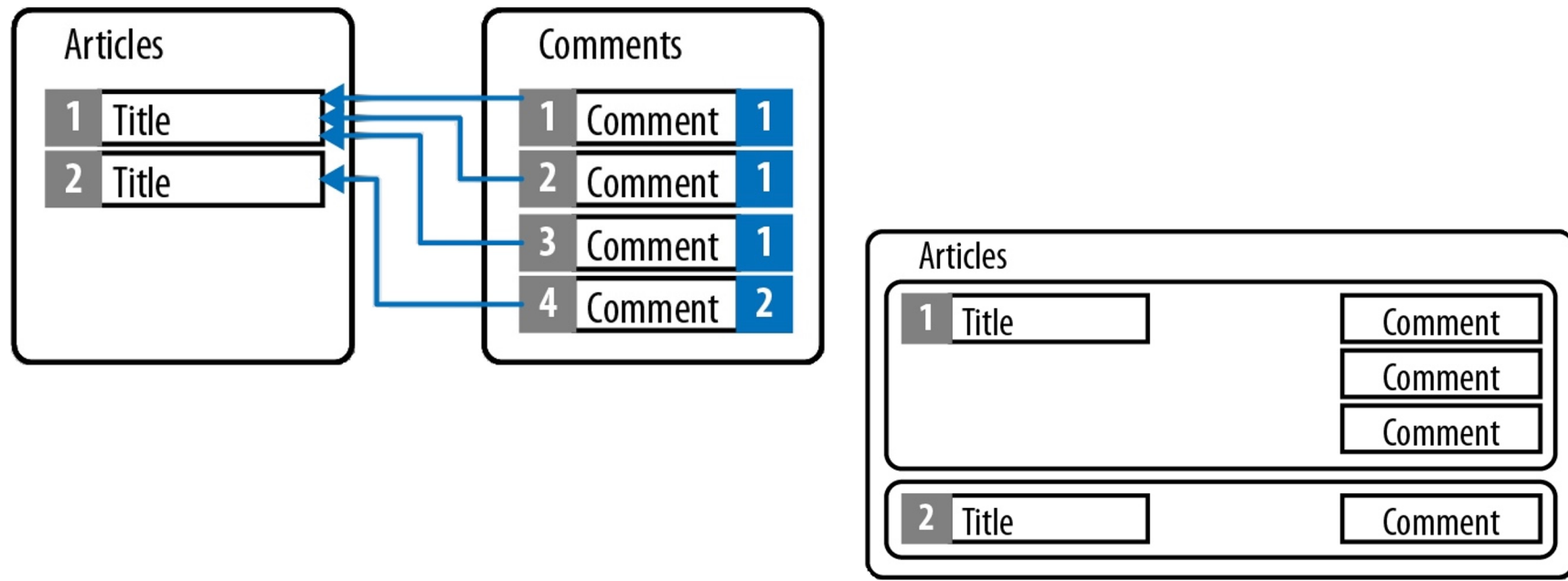
Row 1	US
	Alpha
	3.000
Row 2	US
	Beta
	1.250
Row 3	JP
	Alpha
	700
Row 4	UK
	Alpha
	450

Column Store

Country	US
	US
	JP
Product	UK
	Alpha
	Beta
Sales	Alpha
	Alpha
	3.000
	1.250
	700
	450

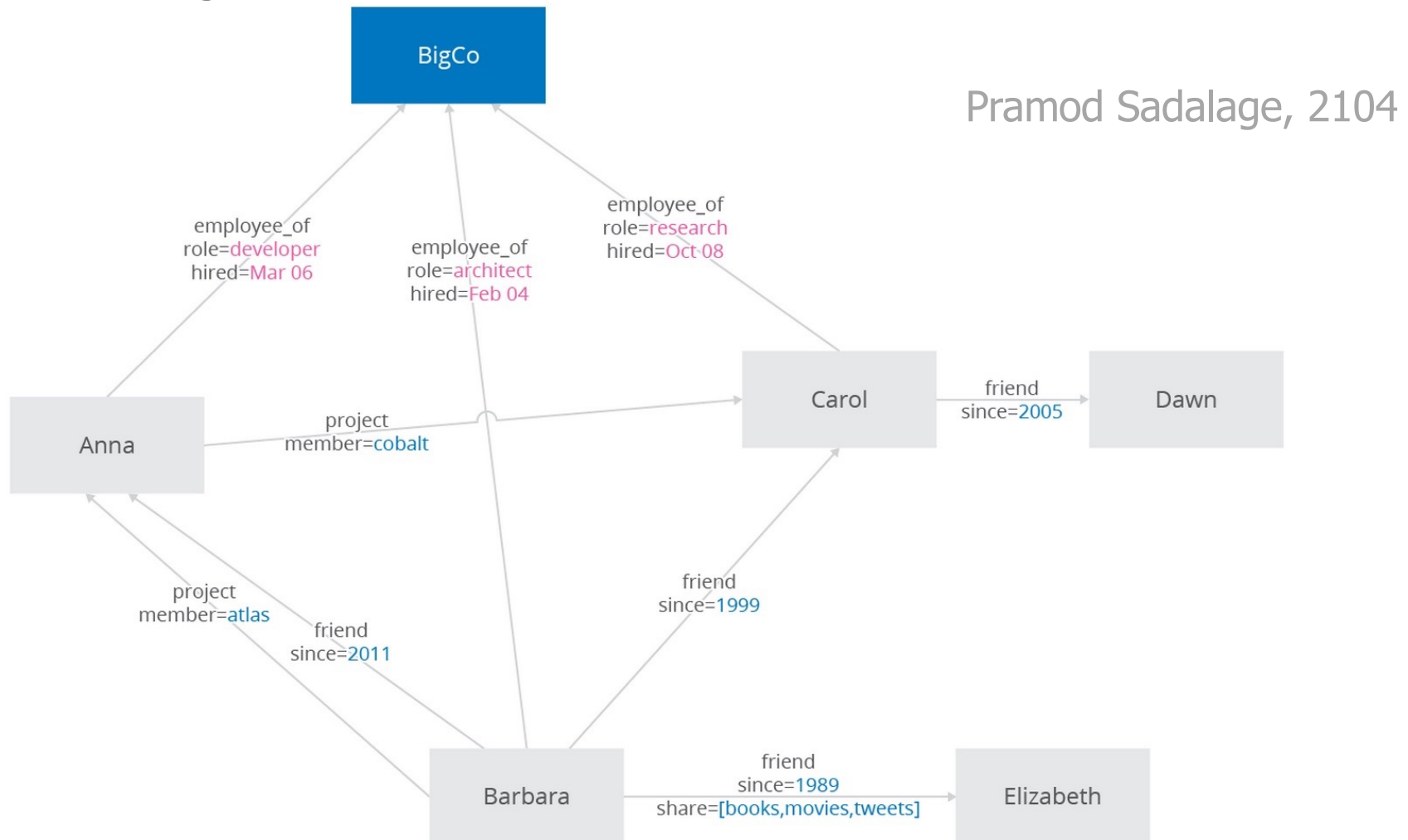
Document-oriented

- Data are stored as
 - Key
 - Complex data structure ~ document
 - Key-value pairs, key-array pairs, nested documents



Graph

- Storing information about networks



Query languages for NoSQL DB

- UnQL (Unstructured Query Language)
 - Couchbase, SQLite
- CQL (Cassandra Query Language)

